Implementation Plan Review & Suggestions

August 14, 2026

You · Aug 14, 07:43

Attachments: Claude-Critical project idea analysis-20260814-1311.md

check this, especially the last two messages for the final implementation plan, and critcially review it, as well as suggest improvements and a better, more detailed implementation plan

---

GPAI · Aug 14, 07:45

I'll read the file first.

I read the whole conversation, with focus on the final two turns (the structured-output pivot and the resulting "Final — Structured-Output Architecture" blueprint). Note: I only have the transcript, not the repo, so this is a review of the plan, not of code.

Verdict first

The pivot is correct and is the single best decision in the whole thread. Removing internal LLM synthesis:

- kills the only network/API/quota dependency on the critical path,
- makes the tool deterministic and therefore *evaluable* with standard IR metrics,
- aligns with the reference architecture (codanna) you already chose,
- and makes grounding stronger, since retrieved facts and the narration now live in the same agent context instead of being laundered through a second model.

Keeping `--explain` as an opt-in demotion rather than deleting it is also right.

So the direction is good. The problems are in the parts the pivot quietly *created* and the parts that were never de-risked.

Critical problems with the final plan

1. The embedding target changed, and the plan waves at it

The pivot's one honest gap is acknowledged then dropped: without an LLM-authored summary, you now embed "diff hunk + commit message + function signature." That is not a detail — it is the core retrieval quality decision of the entire project, and it is the thing your evaluation will actually measure.

Concrete risks: - Diff hunks are lexically dominated by syntax noise (`}`, `import`, whitespace, renamed variables). all-MiniLM-L6-v2 was trained on natural-language sentence pairs, not on unified diff text. Cosine similarity over raw hunks will be much weaker than over prose. - Commit messages in real repos are mostly `fix`, `wip`, `merge branch 'main'`. High-signal, low-recall. - The user query is natural language ("why was retry logic added"), the document is code-diff text. That's a cross-modal retrieval problem, and MiniLM is a poor bridge for it.

This is the difference between a project that works and one that returns plausible-looking garbage. It must be settled by measurement in week one, not assumed.

Fix: build a *template-rendered pseudo-document* per chunk, deterministically, no LLM:

```
Function `fetch_user` in `api/client.py` was modified by alice on 2021-03-01.
Commit message: add retry on transient network errors.
Added: retry loop, backoff, max_attempts parameter.
Removed: direct requests.get call.
Signature changed: fetch_user(id) -> fetch_user(id, retries=3)
Surrounding docstring: Fetch a user record from the API.
```

Every field there is mechanically derivable (git + tree-sitter + added/removed identifier extraction). It converts the diff into something MiniLM-shaped without an LLM. That single move is probably worth more Recall@10 than any amount of HNSW tuning.

2. The plan has no baseline, so it cannot claim anything

Nowhere in the final blueprint is there a non-semantic baseline. For a defense, "our embedding search gets Recall@10 = 0.68" is meaningless. `git log -S <keyword>` and BM25 over commit messages + diffs are both trivial to implement and are the honest comparison. If BM25 beats your embeddings — which happens more often than people expect on code — you need to know that in week one, not at submission.

Fix: make BM25 (via `tantivy`) a *first-class retrieval path*, not a baseline you discard. Hybrid BM25 + dense with reciprocal rank fusion is nearly free once both exist, and it is defensible, standard, and almost always beats either alone on code. This also gives you an exact-identifier fallback, which dense retrieval is genuinely bad at (searching `fetch_user` should never miss).

3. Evaluation is described as "cleaner" but has no ground truth source

The plan says evaluation collapses to Recall@5/Recall@10/MRR/chain-completeness. Good — but where do the labels come from? Hand-labeling 50 queries against your own tool's output is circular and is the single most likely thing a professor attacks.

Fix, in order of preference: 1. **Linked-issue ground truth.** Pick target repos where commits reference issue/PR numbers. Query = the issue title/body. Ground truth = the commit(s) that closed it, from the git trailer or GitHub API. This gives you *hundreds* of free, non-circular labeled query→commit pairs. This is the strongest methodological move available to you and it costs one afternoon of scripting. 2. **Revert pairs.** A commit that reverts X is ground truth that X is relevant to "what broke". 3. Hand-labeled set (~30 queries) only as a supplement, labeled *before* you look at the tool's output, with the criteria written down.

For `impact` precision, the held-out-history validation from §11 is genuinely good: train co-change stats on commits before date T, evaluate predictions against commits after T. Keep it, and report a baseline (file-level co-change, or "predict the most-frequently-changed files") so the number means something.

4. Chain assembly is the actual differentiator and is under-specified

"Deterministic chain assembly" is what makes this more than a vector-search demo, and it rests entirely on tracking a function's identity across commits — through renames, moves

between files, signature changes, and reindentation. `git log --follow` is file-level and already unreliable; function-level identity tracking is a real research-flavored problem.

The plan asserts chain completeness as a metric without specifying the algorithm. That's the highest-risk unaddressed item in the blueprint.

Fix — specify it explicitly: - Identity key: `(file_path, qualified_name, arity)` as primary, plus a content fingerprint (normalized body token multiset, or a min-hash of statement-level tokens) as secondary. - Per commit, match functions in parent vs child by: exact key match → fingerprint similarity above threshold → `git`'s own rename detection on the file → otherwise treat as new. - Persist a stable `function_lineage_id` so a chain is a simple indexed lookup, not a graph walk at query time. - Chain-completeness metric: on repos where you can reconstruct truth by replaying `git log -L` per function, measure precision/recall of your lineage links. `git log -L` gives you a free oracle here. Use it.

5. Tree-sitter Python only is thinner than it looks

Python-only is a reasonable Tier 0 scope, but it caps your evaluation to Python repos and quietly limits the linked-issue ground truth pool. Also, tree-sitter parses *current* file content; you must parse the *blob at each historical commit*, including commits where the file was syntactically broken. The plan doesn't mention failure handling for unparseable historical blobs — which will happen, and will crash a naive indexer partway through a 20k-commit repo.

Fix: parse failures must be recorded as a degraded chunk (file-level, no function attribution), never fatal. Track a parse-success rate as an indexing health metric and report it.

6. Performance claims are unmeasured

"Flat cosine first, HNSW only if measured" is exactly the right instinct — but there's no stated budget. Without a number, "if measured perf requires it" never triggers.

Fix: write the budgets down now and treat them as tests: - index a 10k-commit repo: < 10 min cold, single machine - incremental index of 1 new commit: < 2 s - `search` p95 latency: < 300 ms - MCP tool call round trip: < 500 ms - DB size: < 100 MB per 10k commits

Flat cosine over ~200k chunks × 384 dims is ~300 MB and ~50 ms per query with SIMD — you almost certainly never need HNSW. Measure and say so; "we proved we didn't need the fancy index" is a great defense slide.

7. `--watch` and MCP are scope you don't need

`--watch` polling HEAD with auto-reindex is a background-process, cache-invalidation, partial-state problem. It buys almost nothing for a demo (agents can call `index` themselves) and can eat a week. Similarly, MCP server *and* CLI *and* skill profiles *and* `npx` packaging is four delivery surfaces for one engine.

Fix: MCP over stdio is the one that matters for the positioning (agent-facing), the CLI is needed for humans and for your own dev loop. Ship those two. `--watch` → Tier 4. `npx` wrapper → nice-to-have, and be careful: shipping a Rust binary through npm means platform-specific optional deps and prebuilt artifacts per target triple. That's a day of CI work, not an afternoon.

8. Secret scanning is a liability, not a feature

A half-built regex secret scanner in a portfolio project invites the question "what's your false negative rate?" and there's no good answer. It also isn't related to your thesis.

Fix: either drop it, or reduce it to *redaction of output* using a small, cited ruleset (high-entropy strings + a handful of well-known key prefixes), explicitly framed as "defense-in-depth for output display, not a security guarantee." Don't claim detection coverage.

9. The `--explain` demotion isn't fully carried through

`generated_summary` is left as a nullable column in `commits`. If summaries only exist for `--explain`, they don't belong in the core schema, and they must never be an embedding input, or you reintroduce nondeterminism into your evaluated path. Also: if a summary is cached from an LLM, it needs a provenance/model/version field or your results aren't reproducible.

Fix: move it to a separate `explain_cache(hash, model, prompt_version, text, created_at)` table. Core schema stays LLM-free. Reproducibility preserved.

Better implementation plan

Phase 0 — Prove the thesis before building the product (2-3 days)

Do this in Python, throwaway, before writing any Rust. The goal is a single number.

1. Pick 3 target repos with good issue-linking and Python code (e.g. `requests`, `flask`, `pydantic` — verify linking density first).
2. Script: harvest linked-issue ground truth → N query→commit pairs.
3. Build three quick retrievers over commit-message+diff text: BM25, MiniLM on raw diff, MiniLM on the templated pseudo-document from fix #1.
4. Report Recall@10 and MRR for each.

Gate: if the templated variant doesn't clearly beat BM25 and raw-diff, you learn that *now*, for three days of cost, and you adjust the thesis (e.g. lean harder on chain assembly and impact, which are graph/stats wins that don't depend on embedding quality at all). This step is what turns the project from "I built a thing" into "I measured a claim."

Phase 1 — Core engine, Rust, offline (1.5-2 weeks)

- `git2` extraction: commits, parents, per-file diffs, rename detection on.
- Historical blob parsing with tree-sitter; graceful degradation on parse failure; parse-success telemetry.
- Chunker producing the templated pseudo-document per (commit, function) touch.
- SQLite schema (see below), `rusqlite`, WAL mode, explicit indices.
- Function lineage resolution (fix #4), validated against `git log -L` oracle.
- `fastembed-rs` MiniLM embeddings → mmap'd f32 array + id map.
- Retrieval: `tantivy` BM25 + flat cosine dense, fused with RRF.
- Deterministic chain assembly by `function_lineage_id`.
- Commands: `index`, `search`, `evolution`, `show`, all `--json` first.

Definition of done: the Phase 0 eval harness runs against the Rust binary and reproduces or beats the Python prototype's numbers.

Phase 2 — Graph and stats layer (4-5 days)

- `petgraph`: commit→file→function→caller edges; co-change edges with support/confidence.
- `trace`, `impact`, `blame`, `stats`.
- Held-out-history validation for `impact`, with a stated baseline.

Phase 3 — Agent surface (4-5 days)

- `rmcp` stdio server: `search_history`, `function_evolution`, `change_impact`, `who_knows_this`. Typed structured returns, identical shapes to `--json`.
- One consolidated `SKILL.md` + `gitctx init` writing skill profile and AGENTS.md stanza.
- Human `--format table` rendering (this is the gap the pivot opened — budget real time for it; a wall of JSON in a terminal is a bad demo).

Phase 4 — Evaluation, docs, defense artifacts (4-5 days)

- Full eval report: retrieval table (BM25 / dense / hybrid, ablated), lineage precision/recall, impact precision vs baseline, perf against the stated budgets, parse-success rates.
- Failure analysis section. Explicitly list what it gets wrong. This reads as maturity and pre-empts the harshest questions.
- README with the positioning line, an asciinema demo, and the architecture diagram.

Phase 5 — Optional, only if margin exists

`--explain` + `LlmProvider` + Gemini, `--watch`, `npx` wrapper, second language (TypeScript grammar), secret redaction.

Revised core schema sketch

```
CREATE TABLE commits (
  hash TEXT PRIMARY KEY, author TEXT, email TEXT,
  authored_at INTEGER, message TEXT,
  parent_hashes TEXT, insertions INTEGER, deletions INTEGER
);

CREATE TABLE function_lineage (
  lineage_id INTEGER PRIMARY KEY,
  current_path TEXT, current_name TEXT, language TEXT
);

CREATE TABLE function_touches (
  touch_id INTEGER PRIMARY KEY,
  lineage_id INTEGER REFERENCES function_lineage,
  commit_hash TEXT REFERENCES commits,
  path TEXT, qualified_name TEXT, arity INTEGER,
  change_type TEXT,            -- added|modified|removed|renamed|moved
  signature_before TEXT, signature_after TEXT,
  fingerprint BLOB,            -- minhash for lineage matching
  chunk_text TEXT,             -- the templated pseudo-document
  parse_status TEXT            -- ok|degraded|failed
```

```
);
CREATE INDEX ix_touch_lineage ON function_touches(lineage_id, commit_hash);

CREATE TABLE embeddings (touch_id INTEGER PRIMARY KEY, row_offset INTEGER);
CREATE TABLE cochange (file_a TEXT, file_b TEXT, support INTEGER, confidence REAL);
CREATE TABLE explain_cache (             -- isolated, optional, never an index input
  key TEXT PRIMARY KEY, model TEXT, prompt_version TEXT,
  text TEXT, created_at INTEGER
);
```

The three things that decide whether this succeeds

1. **Does the templated pseudo-document beat BM25?** Answer it in Phase 0, in Python, before touching Rust.
2. **Can you track function identity across renames reliably?** `git log -L` is your free oracle. Use it to produce a real number.
3. **Is your ground truth non-circular?** Linked issues, not hand-labels.

Everything else in the blueprint — Rust, MCP, npx, skill files, HNSW-vs-flat, secret scanning — is execution detail. Those three are the thesis. The final plan in the transcript is strong on execution detail and silent on all three.

You · Aug 14, 07:51

This is a genuinely strong second opinion — sharper on methodology than anything I flagged myself. Worth going through it honestly before rewriting anything, including the one place it's arguing against a version of the plan that's already been superseded.

Assessment of the critique

Right, and adopted:

- **#1 (embedding target under-specified)** — this is the best catch in the whole review. When the LLM-summary pivot happened, I said "embed the raw diff hunk + message + signature" without examining whether all-MiniLM-L6-v2 (trained on natural-language sentence pairs, not unified-diff syntax) would actually perform on that. The templated pseudo-document fix is exactly right, cheap, deterministic, and directly addresses the cross-modal mismatch between a natural-language query and diff-shaped text. Adopting it.
- **#3 (circular evaluation)** — also right, and I hadn't addressed it anywhere across this whole conversation. "Hand-write 20-30 questions against your own tool's output" is grading yourself with your own rubric. Linked-issue ground truth (query = issue title/body, label = the commit(s) that closed it) is a real, established methodology, not a nice-to-have — adopting it as the primary evaluation source, with revert-pairs as a second free signal.
- **#4 (chain assembly / function-lineage tracking was asserted, not specified)** — correct, and a real gap. "Deterministic chain assembly" only holds if function identity survives renames, and I never actually specified the algorithm. The identity-key-plus-fingerprint approach, with `git log -L` as an independent (imperfect but useful) oracle to validate against, is concrete and buildable. Adopting it.
- **#5 (historical blob parse failures)** — correct and easy to miss until it happens on commit #4,000 of a real run. Adopting graceful degradation + a parse-success-rate health metric.

- **#6 (no performance budgets)** — fair. “Measure before optimizing” only works if there’s a number that triggers the decision. Adopting the stated budgets.
- **#9 (`generated_summary` in the core schema)** — sharp catch. A nullable column is an invitation to accidentally use it as an embedding input later and silently reintroduce nondeterminism, and it has no model/prompt-version provenance. Moving it to an isolated `explain_cache` table is strictly better. Adopting it.
- **Phase 0 (prove the thesis in throwaway Python before committing to Rust)** — this is good engineering judgment, full stop. De-risking the retrieval-quality assumption cheaply before investing in the Rust implementation is the right sequencing, and it fits the “agent removes typing time” theme from earlier in this conversation — a disposable Python prototype is fast to build and cheap to discard if the answer is bad news.

Right, but overstated on effort — correcting the estimate, not the substance:

- **#7 (`--watch` and npx are more work than I credited)** — the critique is right that I undersold both. `--watch` is a background-process/cache-invalidation/concurrent-read problem, not a flag you bolt on; I’ll be honest about that and hold it at low priority rather than “cheap to add.” The npx-wrapper point is also fair — per-target-triple cross-compilation CI (linux-x64, macos-arm64/x64, windows-x64 at minimum) is a day or two of CI configuration, not an afternoon. Correcting both estimates rather than dropping either outright, since neither is core to the thesis and both were already non-Tier-1 in the last blueprint.

Right, needs a note rather than a full point already resolved:

- **#2 (no BM25 baseline)** — this critique is reviewing the “Structured-Output Architecture” blueprint specifically (it says so explicitly), which was two turns before the current one. The very next blueprint after that (when you asked to add codanna’s techniques as core) already made Tantivy BM25 + RRF fusion a **Tier 1 core requirement**, with an explicit hybrid-vs-vector-only ablation in the evaluation section. So the underlying concern was already fixed by a later revision the reviewer didn’t see — but the `git log -S`-as-trivial-baseline suggestion is a good additional, even cheaper sanity check worth adding alongside Tantivy, so I’m folding that in too.

Debatable, adopting the spirit not the letter:

- **#8 (secret scanning is a liability)** — I don’t think the right move is dropping it, but the critique’s own suggested fix (redaction framed explicitly as defense-in-depth, not a detection guarantee, small cited ruleset) is correct and is what I’m adopting. A half-built scanner claiming “we detect secrets” invites a fair “what’s your false-negative rate” question with no good answer; a scanner explicitly scoped as “reduces accidental exposure in tool output, not a security control” doesn’t invite that question, and still demonstrates the same engineering maturity at zero extra cost.

The three things the critique calls out as “the actual thesis” (embedding target, lineage tracking, ground-truth non-circularity) are a genuinely better articulation of project risk than anything in my own evaluation sections up to this point. That reordering — those three first, everything else is execution detail — is worth keeping as the framing for your own planning, not just this response.

Project Blueprint: Git History Intelligence (Final, Post-Review)

0. Positioning — unchanged

Codanna: what the code looks like now. `gitctx`: why it looks that way, and what's historically been risky to change. Now with three explicit, de-risked methodological claims instead of asserted ones: a validated embedding strategy, non-circular evaluation, and a specified (not hand-waved) chain-assembly algorithm.

1. Phase 0 — Prove the thesis before writing Rust (2-3 days, throwaway Python)

New phase, precedes everything else. Goal: one number, decided before committing to architecture.

1. Pick 2-3 target repos with dense issue-linking and Python code (verify linking density before committing — not every repo references issues in commit messages).
2. Script: harvest ground truth via `git log` trailers ("Fixes #N"/"Closes #N") cross-referenced with the GitHub API for issue title/body → N query→commit pairs, non-circular by construction.
3. Build three quick retrievers over the same corpus: BM25 (or a simple TF-IDF stand-in), MiniLM on raw diff text, MiniLM on the templated pseudo-document (below).
4. Report Recall@10 and MRR for each.

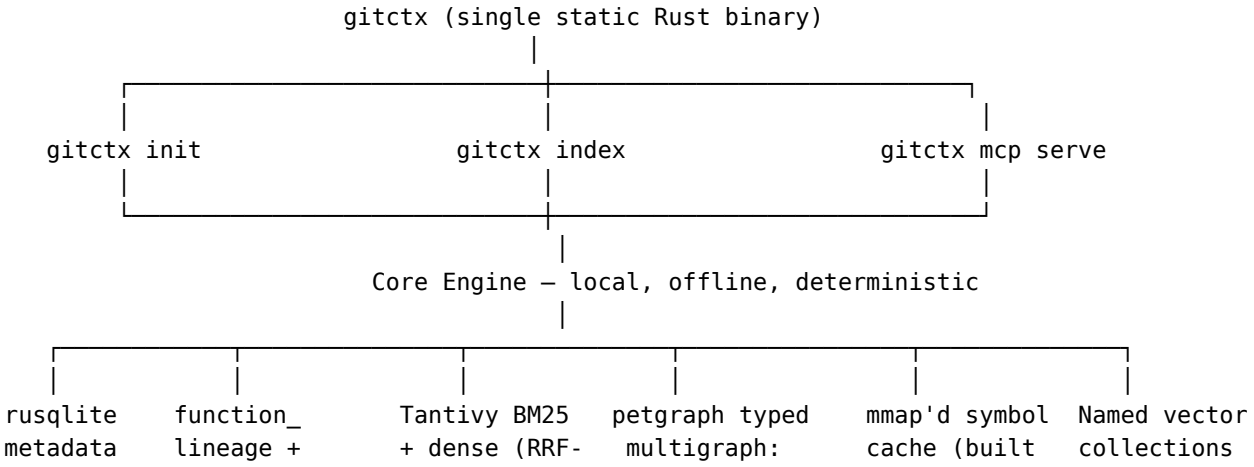
Gate: if the templated variant doesn't clearly beat both raw-diff-MiniLM and BM25, that's decided now, for three days of cost — and it justifies leaning harder on chain assembly and `impact` (graph/stats wins that don't depend on embedding quality at all) rather than discovering this after two weeks of Rust.

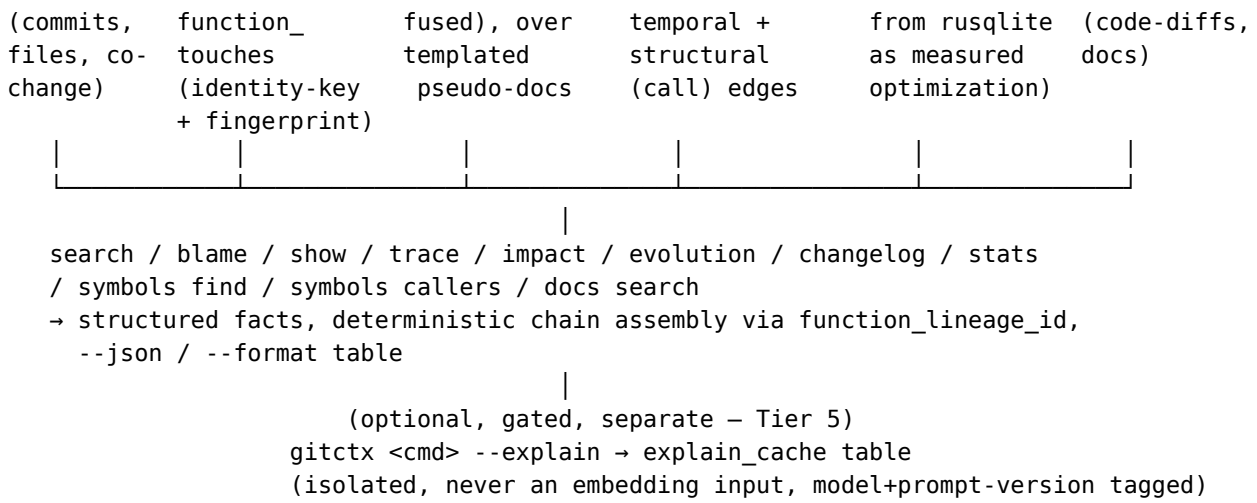
Template-rendered pseudo-document (mechanically derived, no LLM):

```
Function `fetch_user` in `api/client.py` was modified by alice on 2021-03-01.
Commit message: add retry on transient network errors.
Added: retry loop, backoff, max_attempts parameter.
Removed: direct requests.get call.
Signature changed: fetch_user(id) -> fetch_user(id, retries=3)
```

Worth ablating in Phase 0 alongside this: a code-aware embedding model (e.g., a jina-code-style model available via `fastembed-rs`/`candle`) on raw diff text, as a second axis — template-rendering and model choice are two separate levers, and Phase 0 is cheap enough to test both before deciding.

2. Architecture — updated





3. Data model — revised per #4 and #9

```
CREATE TABLE commits (
    hash TEXT PRIMARY KEY, author TEXT, email TEXT,
    authored_at TIMESTAMP, branch TEXT, parent_hashes TEXT,
    message TEXT, insertions INTEGER, deletions INTEGER
    -- generated_summary REMOVED from this table — see explain_cache
);

CREATE TABLE files (id INTEGER PRIMARY KEY, path TEXT, language TEXT);

-- NEW: identity persists across renames/moves, separate from any one touch
CREATE TABLE function_lineage (
    lineage_id INTEGER PRIMARY KEY,
    current_path TEXT,
    current_qualified_name TEXT,
    language TEXT
);

-- NEW: replaces commit_function_changes; one row per (commit, function) touch
CREATE TABLE function_touches (
    touch_id INTEGER PRIMARY KEY,
    lineage_id INTEGER REFERENCES function_lineage(lineage_id),
    commit_hash TEXT REFERENCES commits(hash),
    path TEXT, qualified_name TEXT, arity INTEGER,
    change_type TEXT,              -- added|modified|removed|renamed|moved
    signature_before TEXT, signature_after TEXT,
    fingerprint BLOB,               -- minhash of normalized body tokens, for
                                    -- lineage matching across renames
    chunk_text TEXT,                -- the templated pseudo-document (§1)
    parse_status TEXT               -- ok|degraded|failed — see §5
);
CREATE INDEX ix_touch_lineage ON function_touches(lineage_id, commit_hash);

CREATE TABLE commit_file_changes (
    commit_hash TEXT REFERENCES commits(hash),
    file_id INTEGER REFERENCES files(id),
    change_type TEXT, insertions INTEGER, deletions INTEGER
);

CREATE TABLE co_change_stats (
    file_a INTEGER REFERENCES files(id), file_b INTEGER REFERENCES files(id),
```

```
    co_change_count INTEGER, followup_fix_count INTEGER,
    PRIMARY KEY (file_a, file_b)
);

CREATE TABLE call_edges (
    caller_lineage_id INTEGER REFERENCES function_lineage(lineage_id),
    callee_lineage_id INTEGER REFERENCES function_lineage(lineage_id),
    resolved_at_commit TEXT REFERENCES commits(hash),
    resolution_kind TEXT,           -- intra_file | cross_file
    PRIMARY KEY (caller_lineage_id, callee_lineage_id)
);

-- NEW: isolated, optional, never an embedding input, fully reproducible
CREATE TABLE explain_cache (
    cache_key TEXT PRIMARY KEY,     -- e.g. commit_hash or lineage_id+commit_hash
    model TEXT, prompt_version TEXT,
    text TEXT, created_at TIMESTAMP
);

CREATE TABLE index_state (key TEXT PRIMARY KEY, value TEXT);
```

4. Function-lineage tracking — the specified algorithm (fixes #4)

Per commit, match functions between parent and child versions of a file by, in order: 1. **Exact key match**: `(file_path, qualified_name, arity)` unchanged → same lineage. 2. **Fingerprint match**: normalized-token minhash similarity above a threshold, for same-file signature edits. 3. **Git's own rename detection** on the file, cross-checked against fingerprint similarity, for moved/renamed files. 4. Otherwise: new lineage.

Validation, per the critique's suggestion: on a sample of functions, compare your lineage chains against `git log -L :funcname:file` (git's own, independent, heuristic function-history tracer) as an approximate oracle. Report precision/recall of your lineage links against it — an honest number, not an assumed one. `git log -L` itself isn't perfect ground truth (it's regex-based), so frame this as "agreement with an independent heuristic," not "validated against truth" — still a real, reportable result.

5. Historical parsing — graceful degradation (fixes #5)

Tree-sitter parses the blob *at each historical commit*, not just current content — including commits where a file was mid-edit and syntactically broken. A parse failure produces a **degraded chunk** (file-level attribution, no function boundary) rather than crashing the indexer. Track and report `parse_status` distribution (`ok`/`degraded`/`failed`) as an indexing health metric across the whole run — worth a line in the eval report, since a high degraded-rate on a real repo is itself an honest, interesting finding.

6. Retrieval and evaluation — revised per #1, #2, #3, #6

- **Retrieval:** Tantivy BM25 + dense (over templated pseudo-documents, per Phase 0's winning config) fused via RRF ($k \approx 60$) → deterministic chain assembly by `lineage_id`, chronological, complete.

- **Ground truth (fixes #3):** primary source is linked-issue pairs (harvested the same way as Phase 0, at larger scale); revert-pairs as a secondary free signal; a small hand-labeled supplement (~20-30 questions) only where linked-issue coverage is thin, labeled *before* looking at tool output, criteria written down in advance.
- **Metrics reported:** Recall@5/10, MRR, chain-completeness (via §4's validation), `git log -S` and BM25-alone as baselines alongside hybrid (fixes #2's spirit even though Tantivy/RRF itself was already core in the prior revision).
- **`impact` precision:** unchanged — held-out-history validation, now explicitly reported against a stated baseline (e.g., "predict the most-frequently-changed files") so the number is interpretable, not just present.
- **Performance budgets (fixes #6), treated as pass/fail tests, not aspirations:**

Metric	Budget
Cold index, 10k commits	< 10 min, single machine
Incremental index, 1 commit	< 2 s
`search` p95 latency	< 300 ms
MCP round trip	< 500 ms
DB size	< 100 MB per 10k commits

7. Current-state features — unchanged, carried forward as core

Multi-language tree-sitter extraction (Python/Rust/JS-TS), call graph construction (now backed by `function_lineage`/`call_edges` per §3-4, which strengthens rename-robustness for call graphs too, not just temporal chains), Tantivy hybrid search (already core, validated above), mmap'd symbol cache (built and benchmarked *after* the SQL baseline, per the original sequencing), doc-RAG via named vector collections. All from the prior "Unified Temporal + Current-State" blueprint, unaffected by this review — the reviewer was looking at an earlier turn and didn't see this layer.

8. Secret handling — reframed (fixes #8)

Regex + entropy-based redaction of diff text before storage/embedding, explicitly documented as *"defense-in-depth for output display, not a security guarantee or detection benchmark."* No claimed detection rate. Cheap, still demonstrates the instinct, doesn't invite a question with no good answer.

9. Implementation order — revised

Phase 0 (2-3 days). Throwaway Python: prove templated-pseudo-document retrieval beats BM25 and raw-diff-MiniLM, on linked-issue ground truth. Gate: don't proceed to Rust until this returns a real number.

Phase 1 (1.5-2 weeks) — Core engine, Rust, offline. `git2` extraction with rename detection on; historical blob parsing with graceful degradation (§5); `function_lineage`/`function_touches` (§3-4), validated against `git log -L`; `fastembed-rs` embeddings over templated chunks; Tantivy BM25 + dense + RRF; deterministic chain assembly. Commands: `index`, `search`, `evolution`, `show`,

`--json` first. **Definition of done:** the Phase 0 eval harness runs against the Rust binary and reproduces or beats the Python prototype's numbers.

Phase 2 (4-5 days) — Graph and stats. `petgraph` typed multigraph (temporal + call edges), `co_change_stats`, `trace`, `impact`, `blame`, `stats`. Held-out-history validation for `impact` against a stated baseline.

Phase 3 (1-1.5 weeks) — Current-state layer. Multi-language extraction (+Rust, +JS/TS), call graph construction on top of `function_lineage`, `symbols find`/`symbols callers`, doc-RAG (`docs search`/`add-collection`). Call-graph accuracy sample (~30-50 edges hand-verified, `intra_file` vs `cross_file` precision reported separately).

Phase 4 (4-5 days) — Agent surface. `rmcp` stdio MCP server, skill profiles (`.agents/skills` + `.claude/skills`), `gitctx init`/`AGENTS.md` stanza, `--format table` human rendering (real budgeted time here — a wall of JSON is a bad demo). MCP + CLI are the two delivery surfaces that matter; `npx`/`--watch` explicitly deferred to Phase 6.

Symbol cache (1-2 days, within Phase 3 or 4). Benchmark SQL baseline at target repo size first, *then* build the mmap'd cache, report both numbers against the §6 budgets.

Phase 5 (4-5 days) — Evaluation, docs, defense artifacts. Full report: retrieval table (BM25/dense/hybrid, ablated, against baselines), lineage precision/recall vs. `git log -L`, `impact` precision vs. baseline, parse-success rates, performance against budgets, call-graph precision by resolution kind. **A failure-analysis section, explicit about what it gets wrong** — reads as maturity, preempts the harshest defense questions. README with the positioning line and architecture diagram.

Phase 6 — Optional, only with runway remaining. `--explain` + `explain_cache` (Gemini, gated), `--watch` (now honestly scoped as a real background-process problem, not a quick flag), `npx` wrapper (now honestly scoped as a day or two of cross-compilation CI, not an afternoon), 4th language, MCP-sampling adapter, validated ML risk model beyond co-change heuristic.

10. The three things that actually decide whether this succeeds

Keeping the reviewer's framing, because it's a better summary than anything I'd written myself:

1. **Does the templated pseudo-document beat BM25 and raw-diff embeddings?** Answered in Phase 0, before Rust.
2. **Can function identity be tracked across renames reliably?** Answered against the `git log -L` oracle, with a real precision/recall number.
3. **Is the evaluation ground truth non-circular?** Answered by building on linked issues and reverts, not hand-labels graded against your own tool's output.

Everything else — Rust, MCP, npx, skill files, HNSW-vs-flat, secret handling, multi-language breadth — is real, valuable execution work, but it's no longer being asked to carry the thesis. These three are.

GPAI · Aug 14, 07:54

Let me check one factual claim against the transcript before responding.

First, a correction I owe you: on **#2 you are right and I was wrong for a boring reason** — I sampled the transcript rather than reading it linearly, and the Tantivy/RRF/symbols/doc-RAG revision sits later in the file than the "Structured-Output Architecture" heading I anchored on. Grepping it now confirms Tantivy, RRF, `symbols find`, `docs search`, `call_edges`, and the typed multigraph all appear in later turns. That also means my #7 and #8 were partly aimed at stale scope. Discount those two accordingly; #1, #3, #4, #5, #6, #9 and Phase 0 stand.

The revision is a real improvement. Below is what's still load-bearing and unaddressed, roughly in order of how likely each is to actually bite you.

A. Phase 0 has a validity problem, not just a threshold problem

This is the most important remaining issue, and it's subtle enough that adopting my #3 as written could make things *worse* — a rigorous measurement of the wrong thing.

Query distribution mismatch. Linked-issue queries are bug-report-shaped: "TimeoutError raised when connection pool exhausted." Your product's pitch is why-shaped: "why was retry logic added to `fetch_user`." Those are different retrieval tasks with different difficulty profiles. If you optimize the templated pseudo-document against issue-body queries, you may be tuning for a distribution your users never issue.

Lexical leakage inflating the baseline. The commit that closes issue #123 very often restates the issue title, and contains the literal string `Fixes #123`. If that text is in your indexed chunk, BM25 gets a nearly free win from trivial overlap, and the comparison is meaningless — you'll conclude "BM25 beats embeddings" when what you measured was string matching against a leaked label.

Label incompleteness. The closing commit is *a* relevant commit, not *the complete relevant set*. A follow-up fix, the original introducing commit, and the test commit are all legitimately relevant and unlabeled. Recall@10 will therefore be a **lower bound**, and improvements can look flat when the system is actually returning better-but-unlabeled results.

Concrete fixes, all cheap:

- **Strip leakage before indexing:** remove `Fixes/Closes/Refs #N` trailers and issue URLs from `chunk_text`. State that you did.
- **Two query sets, reported separately.** Set A: linked-issue queries (large N, non-circular, automated). Set B: 25-30 why-shaped queries, authored from reading the *issue/PR discussion only*, never from tool output, with labels taken from the issue's linked commits. Set B is small but measures the actual product. Report both; never average them into one number.
- **Frame metrics as lower bounds** and, for the top-5 misses per config, manually check whether the "miss" was actually a relevant unlabeled commit. Report that adjudicated rate. This one paragraph in your eval section will do more for a defense than another 200 queries.
- **Make the gate a decision rule, not a vibe.** With N≈150-300 queries, differences under ~5 points of Recall@10 are noise. Use paired comparison on the same queries plus a bootstrap 95% CI on the difference, and write the rule down in advance: *"proceed with templating if ΔRecall@10 vs. raw-diff $\geq$ 0.05 with CI excluding 0; if templating loses to BM25 alone, dense retrieval is demoted to a secondary signal and the thesis shifts to lineage + impact."*

- **Define the retrieval unit now.** You retrieve `function_touches`; labels are commits. Specify max-pooling: a commit is retrieved at rank *r* if its best-ranked touch is at *r*, after dedupe. Otherwise a commit touching 40 functions floods your top-10 and every metric is garbage.

B. The `git log -L` oracle is blind exactly where your algorithm is novel

`git log -L :func:file` follows a line range within **one file path**, and needs the function to be resolvable at the starting revision. It therefore cannot validate:

- cross-file moves (your §4 step 3 — the hardest and most interesting case),
- functions deleted before HEAD,
- overloads/nested/decorated definitions where its regex heuristic just fails.

So it agrees with you on the easy cases and is silent on the hard ones. Reporting only "agreement with `git log -L`" will produce a high number that means little, and a sharp examiner will spot it.

Add synthetic mutation testing as a second, true oracle. This is cheap and gives you real ground truth on the hard case:

1. Take a real repo, snapshot it.
2. Programmatically generate synthetic commits that apply known transformations: rename function, move function to another file, change arity, reindent/reformat, rename local variables, split one function into two, plus decoy commits that delete a function and add an unrelated same-named one.
3. You know the true lineage by construction. Report recovery precision/recall **per transformation class**.

A table of "rename: 0.98 / move-across-file: 0.81 / split: 0.42 / same-name-different-function: 0.94 rejected" is a far stronger result than a single blended agreement score, and it tells you which step of §4 to invest in.

Two related points: - **Tune the fingerprint threshold on held-out data**, not by eye — otherwise it is exactly the kind of asserted number you just spent a revision eliminating. The synthetic set gives you a tuning split for free. - **Report chain-level, not just link-level, accuracy.** Lineage is transitive; a 0.97 per-link rate over a 30-commit chain is ~0.40 chain-correctness. Since chain completeness is your headline claim, measure it at the chain level.

C. Merge commits and non-linear history are unaddressed

Nothing in the blueprint says how you traverse. This is a silent correctness bug that will corrupt three things at once: diffs (a merge has multiple parents — diff against which?), co-change statistics (merges double-count every change already counted on the branch), and chains (the same logical change appears twice).

Decide explicitly and document it: default to **first-parent traversal**, skip merge commits for diff extraction and co-change accounting unless the merge is a genuine conflict resolution (non-empty diff against *both* parents), in which case index it as its own change. Add `is_merge` and `traversal = first_parent` to `index_state`. Also handle: squash-merge repos (history is linear but each commit is a whole PR — chunk granularity gets coarse; note it), and shallow/grafted clones (refuse to index, or warn).

D. Co-change statistics as specified will be dominated by noise

`co_change_count` as a raw integer is the classic mistake. A single `black`-reformat commit touching 900 files contributes ~400k pairs and swamps every genuine coupling. Also `followup_fix_count` has no definition anywhere.

- **Filter by commit size**: exclude commits touching more than N files (N≈25, tuned and reported) from co-change accounting. Record how many commits were excluded.
- **Store a normalized measure**, not just counts: support, confidence (`P(B|A)`), and lift. Rank by confidence with a minimum support floor.
- **Define "fix"** operationally: (a) commit is linked to an issue labeled bug, (b) commit is a revert of the earlier commit, or (c) message matches a stated regex. Prefer (a)/(b); (c) alone is weak and should be labeled as such.
- **Time-decay** is worth one line: coupling from 2014 is weaker evidence than 2025. Even a simple half-life weighting is defensible and easy.
- Prior art check: `code-maat` (already in your sources) does exactly this and is the natural baseline to cite and beat/match on `impact`. Comparing against a known tool is much stronger than comparing against "most-frequently-changed files."

E. Schema issues that will force a migration later

1. **`call_edges` PK `(caller, callee)` drops time.** You cannot answer "who called this at commit X," and the PK collides when an edge appears, disappears, and reappears. Make it `(caller_lineage_id, callee_lineage_id, valid_from_commit)` with a nullable `valid_to_commit`, or store per-commit rows and compress later. Given `impact` is history-aware, temporal validity is not optional.
2. **`function_lineage.current_path` / `current_qualified_name` are mutable denormalized state** that will drift out of sync on reindex. Derive them from the latest `function_touches` row via a view, or accept them only as a cache with a rebuild step.
3. **`arity` is a weak identity component in Python**: defaults, `*args/**kwargs`, decorators, and `self` make it unstable and coarse. Prefer a normalized signature string plus a separate `param_count`, and don't let arity alone break a match.
4. **`qualified_name` needs a stated convention** for methods, nested functions, closures, conditionally-defined functions, and same-name definitions in different classes. Write the convention down; it's the primary key of your whole identity scheme.
5. Add a **`schema_version`** row in `index_state` and a refusal-to-open path on mismatch. You will change this schema.

F. The scope tension your own framing exposes

§10 says three things carry the thesis. §7 then carries forward a full current-state code-intelligence layer — multi-language extraction, call graphs, symbol cache, and doc-RAG — none of which serve those three, and most of which is a reimplementation of codanna, the tool you're positioning *against* as complementary.

That's the original scope-creep critique returning in a new outfit. Specifically:

- **Doc-RAG (`docs search`, collections) has no relationship to the thesis.** It is a separate product. Cut it, or move it behind Phase 6. It is the single highest-cost/lowest-defense-value item in the plan.

- **Cross-file call resolution in Python is genuinely hard** — dynamic dispatch, `import x as y`, re-exports, monkeypatching, duck typing. 1-1.5 weeks for "multi-language extraction + call graph + doc-RAG" is the most optimistic estimate in the document by a wide margin. Scope it honestly: **intra-file calls plus same-package import resolution only**, everything else marked `unresolved`, and report `intra_file` / `cross_file` / `unresolved` proportions. An honest coverage number beats a silently-wrong graph.
- **Third language (JS/TS) buys breadth you can't evaluate.** Your ground truth pipeline, oracles, and eval corpora are Python-shaped. Adding a language multiplies implementation and evaluation work while adding zero evidence for the thesis. Do it last, or do two languages and say why.

Blunt version: Phases 0-2 plus Phase 5 constitute a defensible, evaluated, novel project. Phase 3 is a second project bolted on. If the calendar slips — and it will — Phase 3 is what should absorb the slip, and the plan should say so now rather than discovering it under pressure.

G. Missing engineering items that are cheap and disproportionately credible

- **Pinned evaluation corpus.** Commit the exact repo URLs *and* commit SHAs used for evaluation, plus the harvested query/label set, into the repo as a versioned dataset. Without pinned SHAs your numbers are unreproducible the moment upstream moves. This costs 20 minutes and is the difference between "results" and "anecdotes."
- **Determinism test.** Index the same repo twice; assert identical `function_touches` rows, identical lineage assignments, and identical top-10 for a fixed query set. You are claiming determinism as a core property — test it, and say you test it. Also pin the embedding model version and hash it into `index_state`.
- **Incremental-vs-full-reindex equivalence test.** Index commits 1..N fully, versus 1..N-10 then incrementally adding 10; assert the DBs match. This is the bug class that silently ruins incremental indexers.
- **Failure taxonomy, pre-registered.** Before you look at results, define the buckets you'll classify misses into (query-vocabulary mismatch, chunk too coarse, lineage break, commit genuinely uninformative, label incomplete). Then report counts. This turns your failure-analysis section from apology into method.
- **Explicit non-goals section** in the README: not a replacement for `git blame`/`git log`, not a security tool, not a code-search tool for current state, no cross-repo support. Pre-empts half the questions.

H. Schedule and gates

The listed phases sum to roughly 6-7 weeks with zero slack, on a plan whose two hardest technical problems (lineage across moves, cross-file call resolution) sit in Phases 1 and 3. Add **20-25% buffer explicitly**, and pre-commit to what gets cut, in order: doc-RAG → third language → symbol cache → cross-file call resolution → `--watch`/`npx`.

Add kill/pivot criteria alongside your existing Phase 0 gate:

Checkpoint	Measure	Pivot if
End Phase 0	ΔRecall@10, templated vs. BM25	Templating loses → demote dense to secondary signal; thesis becomes lineage + impact
Phase 1 midpoint	Lineage recall on synthetic move class	< 0.6 → restrict claim to intra-file lineage, state it, stop investing
End Phase 1	Cold-index time, 10k commits	> 3x budget → reduce corpus size rather than optimize; note the limit
End Phase 2	`impact` precision vs. code-maat baseline	Not better → reframe as "reproduces established co-change findings in a new interface," which is still honest and publishable-adjacent
End Phase 3	`cross_file` resolution coverage	< 50% → ship intra-file only, report coverage

The short version

Your revision fixed the right things. What remains, ranked:

1. **Phase 0 measures a different query distribution than the product serves, and leaks labels into the baseline.** Fix leakage, split into two query sets, treat metrics as lower bounds, pre-register the decision rule.
2. **`git log -L` can't validate cross-file moves** — the case that makes §4 interesting. Add synthetic mutation testing for true per-transformation ground truth.
3. **Merge/non-linear history is undefined** and will silently corrupt diffs, co-change, and chains. First-parent by default, documented.
4. **Co-change needs size filtering and normalization** (support/confidence/lift), a real definition of "fix," and code-maat as the baseline.
5. **`call_edges` needs temporal validity**; identity keys need a written convention; add `schema_version`.
6. **Phase 3, especially doc-RAG, doesn't serve the thesis** and is where the schedule should absorb slip. Say that now.
7. **Pin the eval corpus by SHA and test determinism/incremental equivalence** — cheapest credibility in the entire plan.

You · Aug 14, 14:37

Based on all this, as well as other issues/pitfalls you can think of, generated a fully detailed and elaborated-upon step-by-step implementation plan, with all possible issues properly mentioned, along with possible solutions to try/check, as well as how to verify everything

Based on all this, as well as other issues/pitfalls you can think of, generated a fully detailed and elaborated-upon step-by-step implementation plan, with all possible issues properly mentioned, along with possible solutions to try/check, as well as how to verify everything

---

GPAI · Aug 14, 14:41

gitctx — Full Implementation Plan (Post-Review, Execution-Ready)

This plan supersedes all prior blueprints. It is organized so that every step has: what you build, what will go wrong, what to try, and how you verify it is correct. Nothing is asserted without a verification method.

Reordering decision baked in: the current-state code-intelligence layer (multi-language, call graphs, doc-RAG) moves *after* the agent surface and evaluation, because it does not carry the thesis and must be the slip absorber. The agent surface moves earlier because it is what makes the project demonstrable.

Part 0. Invariants — decide once, enforce mechanically

These are project-wide rules. Violating any of them silently invalidates your evaluation, so each gets a test, not just a paragraph.

#	Invariant	Enforcement
I1	The indexed/evaluated path never calls a network service or an LLM	CI test runs `index` + all queries with network disabled (`unshare -n` or a deny-all HTTP mock); build feature `explain` off by default
I2	Indexing is deterministic: same repo + same config → byte-identical logical state	Double-index test comparing normalized dumps (Part 6.2)
I3	Incremental index ≡ full index for the same commit range	Equivalence test (Part 6.3)
I4	Every returned fact carries a verifiable citation (commit SHA + path + line range)	Schema NOT NULL constraints; JSON schema validation in tests
I5	No LLM-derived text is ever an embedding input or a ranking signal	`explain_cache` is a physically separate table; grep-based CI lint forbidding `explain_cache` in retrieval modules
I6	Evaluation corpus is pinned by commit SHA	`eval/corpus.lock.toml`; harness refuses to run if repo HEAD ≠ pinned SHA
I7	Every reported number has a stated baseline and a stated uncertainty	Eval report template with required columns; reviewer checklist
I8	Traversal semantics are explicit and recorded in the index	`index_state.traversal = first_parent`, `schema_version`, model hash

Configuration hashing. Compute `config_hash = H(schema_version, embedding_model_id, model_file_sha256, chunk_template_version, tokenizer_version, traversal_mode, cochange_max_files, fingerprint_params)` and store it in `index_state`. Refuse to serve queries if the binary's `config_hash` differs from the index's. This single mechanism prevents the most insidious class of bug in this project: comparing numbers produced under silently different configurations.

Part 1. Phase −1 — Foundation and corpus selection (2-3 days)

Do this before any retrieval work. Corpus choice constrains everything downstream, and a bad choice is discovered painfully late.

Step 1.1 Repository scaffolding

- Cargo workspace, even though you ship one binary. Crates: `gitctx-core` (extraction, chunking, lineage), `gitctx-index` (sqlite, tantivy, vectors), `gitctx-query` (retrieval, chains, stats), `gitctx-cli`, `gitctx-mcp`, `gitctx-eval`. Rationale: the eval harness must link the real

engine, not a copy; a workspace makes that free and prevents drift between "what I evaluated" and "what I ship."
- `xtask` pattern for repeatable dev commands (`cargo xtask eval`, `cargo xtask bench`).
- CI from day one: `fmt`, `clippy -D warnings`, `test`, and a smoke `index` over a tiny fixture repo. Cheap now, unaffordable to retrofit.
- **Fixture repos as test data.** Build 3-5 tiny synthetic git repos programmatically (a `tests/fixtures/mkrepo.rs` helper that shells out to `git`) covering: linear history, a rename, a cross-file move, a merge with conflict resolution, a squash-merge shape, a file that is syntactically broken at one commit. Every unit test runs against these, in milliseconds, offline. This is the highest-leverage two days in the plan — most of the bugs listed later become one-line regression tests.

Step 1.2 Corpus selection with measured criteria

Do not pick repos by reputation. Script the selection.

For ~15 candidate Python repos, compute and tabulate:

- commit count, and count after first-parent filtering
- fraction of commits containing an issue/PR reference (`#\d+`, `Fixes|Closes|Resolves`)
- fraction of *closed issues* that are traceable to ≥1 commit
- merge-commit fraction; squash-merge signature (are most commits PR-shaped with `(#1234)` in the subject?)
- mean/median files per commit; count of commits touching >25 files (reformat storms)
- fraction of `.py` files, presence of vendored/generated directories
- revert-pair count
- rename/move density (`git log --diff-filter=R --find-renames`)

Selection rule, written before you look: you need ≥3 repos where traceable-issue count ≥ 150, merge fraction < 40%, and squash-merge is not universal. Pick a *deliberate mix*: one small (~2k commits), one medium (~15k), one large (~50k+) to expose scaling behavior, and prefer variation in commit-message discipline so you do not overfit to one project's conventions.

Pitfall — squash-merge repos. If every commit is an entire PR, your function-level chunking becomes coarse and your "why" signal is diluted across many functions. Detect it; if a candidate is squash-only, either drop it or keep it explicitly as the "hard case" and report it separately.

Pitfall — vendored code and generated files. `node_modules`, `vendor/`, `_pb2.py`, minified assets, and lockfiles will dominate churn statistics and pollute embeddings. Build a path exclusion list (with a `.gitctxignore`, gitignore syntax) and *report the exclusion fraction*. Also handle: files over a size threshold, binary files, and files with pathological line counts.

Verification: commit the candidate table to `eval/corpus_selection.md` with the SHA of the selection script. Pin chosen repos in `eval/corpus.lock.toml` as `{url, sha, harvest_date}`.

Step 1.3 Ground truth harvesting (the asset the whole project rests on)

Build `eval/harvest.py` producing a versioned JSONL dataset.

Sources, in priority order:

1. **Issue-closing links.** Parse commit messages and merge-commit bodies for `(Fixes|Closes|Resolves|Ref) #N`; also read the GitHub API for issue→closing-commit events (more reliable than message parsing alone, since the platform records it). Query text = issue title + body. Labels = the linked commit(s).
2. **Revert pairs.** `git log --grep '^Revert "'` plus the standard `This reverts commit <sha>` body line. Gives you high-confidence "what broke" pairs.
3. **PR-review-derived queries.** If a PR discussion contains a "why did you do X" question with an answer, that is a gold why-shaped query. Harvest a handful manually; they are the most product-representative queries you will get.
4. **Manual why-set (Set B, see Phase 0).** Authored from issue/PR discussion only, before any tool output exists.

Leakage removal, mandatory. Before anything is indexed, strip from `chunk_text`: issue/PR references, issue URLs, `Fixes/Closes` trailers, `Co-authored-by`, `Signed-off-by`, and the bare `(#1234)` suffix squash-merges append. Keep the raw message in the `commits` table for display; only the *embedded/indexed* text is sanitized. Log the number of strips.

Additional leakage vectors people miss: - The issue *body* often quotes a traceback containing the exact file and function name of the fix. That is legitimate signal (a real user query would too), but note it, because it means BM25 will do well for honest reasons. - Bot commits (dependabot, pre-commit.ci) create thousands of near-duplicate low-value pairs. Exclude bot authors and report the count. - The same issue closed by many commits creates label imbalance; cap labels per query and record the distribution.

Dataset schema:

```
{"qid":"gh-req-4821","repo":"psf/requests","query_set":"A",
 "query":"Session.request hangs indefinitely when...", "query_source":"issue_body",
 "labels":[{"commit":"a1b2c3d","label_source":"github_close_event","confidence":"high"}],
 "harvest_sha":"<script sha>","harvest_date":"2026-08-14"}
```

Verification: hand-audit a random 30 pairs. Record the precision of your harvester (what fraction of auto-derived labels a human agrees are the actual fix). If harvester precision is below ~0.85, fix the harvester before it poisons every downstream number. **Report this precision in the final eval section** — it is the honest ceiling on your metrics, and volunteering it reads as rigor.

Part 2. Phase 0 — Prove the retrieval thesis in throwaway Python (3-4 days)

Purpose: answer one question cheaply — *does deterministic template-rendering of diffs make natural-language retrieval work well enough to be the core of the product?*

Step 2.1 Build the ablation grid

Two independent levers, tested as a grid, not conflated:

Lever 1 — document text: - D1: commit message only - D2: raw diff hunk only - D3: message + raw diff (the naive pivot proposal) - D4: templated pseudo-document (below) - D5: templated pseudo-document minus commit message (isolates whether the template or the message carries the signal)

Lever 2 — retriever: - R1: `git log -S` keyword (trivial floor) - R2: BM25 (rank_bm25 in Python is fine at this scale) - R3: all-MiniLM-L6-v2 dense - R4: a code-aware embedding model (e.g. a jina-code or bge-code class model) - R5: RRF fusion of R2 + best dense

Templated pseudo-document, v1 (every field mechanically derivable):

```
Function `fetch_user` in `api/client.py` was modified by alice on 2021-03-01.
Commit message: add retry on transient network errors.
Signature changed from fetch_user(id) to fetch_user(id, retries=3).
Identifiers added: retry, backoff, max_attempts, RequestException.
Identifiers removed: requests.get.
Control flow added: for loop, try/except.
Docstring: Fetch a user record from the API.
Enclosing class: ApiClient.
Change size: +14 -3 lines.
```

Version this template (`chunk_template_version`). Treat wording as a hyperparameter you may revise once, with the revision reported.

Step 2.2 Metrics protocol — specify before running

- **Retrieval unit vs. label unit.** You retrieve function-touches; labels are commits. Use **max-pooling with dedupe**: a commit's rank is its best-ranked touch after collapsing duplicates. Without this, a 40-function commit floods the top-10 and every metric is meaningless.
- **Metrics:** Recall@5, Recall@10, Recall@20, MRR, plus nDCG@10 if you have graded labels. Report per-repo *and* pooled; never only pooled, because one repo will dominate.
- **Two query sets reported separately.** Set A (large, automated, issue-shaped). Set B (25-30 hand-authored why-shaped queries, written from issue/PR discussion only, labeled before any tool output is seen, with criteria recorded in `eval/setB_protocol.md`). Never average A and B into one headline number — they measure different tasks.
- **Metrics are lower bounds.** Labels are incomplete. For the top-5 misses per configuration, manually adjudicate whether the "miss" was actually a relevant unlabeled commit; report the adjudicated rate alongside the raw rate.
- **Statistics.** Paired comparison on identical queries; bootstrap 95% CI on the *difference* (10k resamples). With N≈150-300, treat $\Delta < 0.05$ Recall@10 as noise.

Step 2.3 The gate — a decision rule, written in advance

Copy this into the repo before running anything:

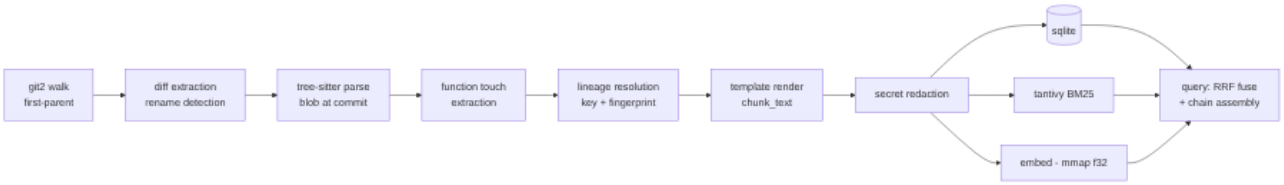
- **Proceed as planned** if D4 beats both D3 and R2-alone by ΔRecall@10 ≥ 0.05 with a CI excluding zero.
- **Proceed with model swap** if D4 + R4 clears the bar but D4 + R3 does not. Cost: verify the code-aware model is available in `fastembed-rs`/`candle` and check its dimensionality and speed against the Part 5 budgets *before* committing.
- **Demote dense retrieval** to a secondary signal if BM25 alone wins or ties. The thesis then becomes lineage + impact + deterministic chain assembly, with hybrid retrieval as plumbing. This is still a strong project; it is only a weak project if you discover it in week six.
- **Reconsider granularity** if all configurations are poor (Recall@10 < 0.3 everywhere). Likely causes to test, in order: retrieval unit too fine (try commit-level documents), corpus quality, leakage-stripping too aggressive, tokenizer truncation (MiniLM's 256-token window silently truncates large hunks — check truncation rates before concluding anything).

Step 2.4 Known Phase 0 traps

- **Truncation.** Measure the fraction of documents exceeding the model's max sequence length. If high, that alone explains poor dense results. Fix by splitting hunks or by putting the highest-signal template lines first.
- **Query-document asymmetry.** Issue bodies are long (hundreds of tokens); documents are short. Test truncating/summarizing queries to their title, and report which works better.
- **Normalization.** L2-normalize embeddings once at write time; cosine then reduces to a dot product. A missing normalization is a classic silent quality bug.
- **Duplicate documents.** Cherry-picks, rebases, and vendored copies create near-identical chunks that crowd the top-k. Deduplicate by content hash at index time and note the collapse rate.

Verification for the whole phase: `eval/phase0_report.md` with the full grid, CIs, truncation rates, adjudication rates, and the gate decision explicitly stated with the number that triggered it. This document is a defense artifact — it proves you made an architecture decision from evidence.

Part 3. Phase 1 — Rust core engine (2-2.5 weeks)



Step 3.1 Traversal and diff extraction

Decisions to hard-code and document: - **First-parent traversal by default.** Merge commits are skipped for diff extraction and co-change accounting *unless* they are genuine conflict resolutions — detectable as a non-empty diff against *both* parents (an "evil merge"). Those get indexed as their own change with `is_merge = 1`. - Enable git rename detection with explicit thresholds (`find_renames`, `find_copies`, similarity threshold recorded in config). - Record `traversal`, `is_merge` handling, and thresholds in `index_state`.

Pitfalls: - **Shallow/grafted clones** silently truncate history. Detect (`is_shallow`, presence of `.git/shallow`) and refuse with a clear error rather than producing wrong stats. - **Orphan branches and multiple roots.** Decide: index `HEAD`'s first-parent line only, by default; offer `--all-branches` later. Document it. - **Non-UTF8 paths and content.** libgit2 gives you bytes. Use lossy conversion for display but hash the raw bytes for identity; do not panic. - **Huge commits** (initial import, vendored drop) will blow memory if you materialize all diffs. Stream per-file, cap per-file diff size, and record truncations. - **Timezone and clock skew.** Commit timestamps can be non-monotonic and even negative-offset. Store both author and committer time as epoch seconds plus offset; order chains by *topological* order first, timestamp second. Never assume time ordering equals ancestry. - **Performance:** libgit2 diffing dominates index time. Parallelize per-commit with `rayon`, but note SQLite writes must be serialized through a single writer (see 3.5).

Verification: fixture-repo tests asserting exact commit/diff counts for linear, merge, evil-merge, rename, and move fixtures. Cross-check total commit count against `git rev-list --first-parent --count HEAD` on a real repo.

Step 3.2 Historical parsing with graceful degradation

Parse the blob **as of each commit**, not the working tree.

- Parse failure or `ERROR` nodes → emit a **degraded chunk** at file level with `parse_status = 'degraded'`; never fatal.
- Track and report the `ok/degraded/failed` distribution as an index health metric.
- Cache parses by blob OID — the same blob recurs constantly across commits, and this is likely a 3-10x index-time win.

Pitfalls: - Python 2 syntax in old history will fail a Python 3 grammar. Expect a degraded band in pre-2015 history; report it rather than hiding it. - Tree-sitter recovers partially from errors; a subtree may be valid even when the file is not. Prefer extracting the functions you *can* resolve, and mark the file `degraded` rather than discarding everything. - Very large generated files can blow parse time/stack. Cap by file size and node count.

Verification: the deliberately-broken fixture must produce exactly one degraded chunk and zero errors. On a real repo, assert `ok` rate above a threshold you record (e.g. > 0.95 post-2015).

Step 3.3 Function identity and lineage — the specified algorithm

Qualified name convention — write this down; it is the primary key of your identity scheme. Specify explicitly for: module-level functions, methods (`Class.method`), nested classes, closures/inner functions (`outer.<locals>.inner`), decorated functions (name from the `def`, decorators recorded separately), conditionally-defined functions (same name, two definitions — disambiguate by definition order index), properties/setters, and `async def`. Record the convention version.

Identity components: `(path, qualified_name, normalized_signature, param_count)` plus a `fingerprint`. Per my earlier note: **arity alone is weak in Python** (defaults, `*args/**kwargs`, `self`, decorators). Use normalized signature as the informative component and never let a bare param-count change break a match.

Fingerprint: MinHash over normalized body tokens — normalize by stripping comments/docstrings, canonicalizing string literals and numbers, and renaming local identifiers to positional placeholders where cheaply possible. Store the signature; keep parameters (`k`, shingle size) in the config hash.

Matching order per commit, per file: 1. Exact identity key match → same lineage. 2. Fingerprint similarity $\geq \tau$ within the same file → same lineage (handles rename-in-place and signature edits). 3. Git rename/copy detection maps old path → new path; then retry (1) and (2) within the mapped file (handles moves). 4. Cross-file fingerprint search *scoped to the commit's touched files* → same lineage (handles moves git did not detect). Scope it; a global search is $O(n^2)$ and will not scale. 5. Otherwise → new lineage.

Ambiguity and conflict rules — do not skip these, they are where correctness dies: - **Splits:** one parent function matching two children. Rule: the child with highest fingerprint similarity inherits the lineage; the other becomes a new lineage with a recorded `derived_from`

edge. Report split detection separately; expect mediocre accuracy and say so. - **Merges (two functions combined into one):** inverse rule; record multiple `derived_from` edges. - **Delete-then-add of the same name** (genuinely different function): must be *rejected* as continuation when fingerprint similarity is low. This is your key false-positive test case. - **Many-to-many within one commit:** resolve as a maximum-weight bipartite matching (greedy by similarity is acceptable; state the choice), never as independent per-function argmax, which produces two functions claiming one lineage. - **Uniqueness constraint:** at most one touch per `(lineage_id, commit_hash)` — enforce in the schema so violations surface as errors, not as corrupt chains.

Threshold tuning: τ must be tuned on a held-out split of the synthetic mutation set (Step 3.4), not chosen by eye. Report the tuning curve.

Step 3.4 Lineage validation — two oracles, because one is blind

Oracle 1 — synthetic mutation testing (true ground truth, and your primary evidence).

Build `gitctx-eval mutate`: take a real repo snapshot, then generate synthetic commits with known transformations:

Class	Transformation	Expected behavior
M1	rename function in place	lineage preserved
M2	move function to another file, unchanged	lineage preserved
M3	move + rename simultaneously	lineage preserved (hardest positive)
M4	change signature (add param w/ default)	lineage preserved
M5	reformat / reindent / add type hints	lineage preserved
M6	rename all local variables	lineage preserved
M7	split one function into two	primary inherits; second recorded as derived
M8	delete function, add unrelated function with same name	lineage **not** continued (false-positive test)
M9	move whole file to new directory	all lineages preserved
M10	copy function into a second location	one lineage continues; copy is new + derived

Report **precision and recall per class**. A table like `M1 0.99 / M2 0.86 / M3 0.71 / M7 0.44 / M8 0.95 rejected` is far more informative and more credible than a single blended score, and it tells you which matching step to invest in.

Oracle 2 — `git log -L` agreement (secondary, with stated blindness). Use `git log -L :func:file` for cross-checking, and state plainly that it (a) cannot follow cross-file moves, (b) requires resolution at the start revision, (c) is regex-heuristic. Frame results as “agreement with an independent heuristic on cases it can express,” never as validation. Report the

fraction of your lineages the oracle *cannot even express* — that number itself argues for why your approach is needed.

Chain-level metric, not just link-level. Lineage is transitive: 0.97 per-link over a 30-link chain is ~0.40 chain-correctness. Since "complete chronological chain" is your headline claim, report exact-chain accuracy, chain-prefix accuracy, and a length-stratified breakdown (chains of 2-5, 6-15, 16+).

Step 3.5 Storage layer

Final schema, with the review's fixes applied:

```
CREATE TABLE index_state (key TEXT PRIMARY KEY, value TEXT);
-- required keys: schema_version, config_hash, traversal, embedding_model_id,
-- embedding_model_sha256, chunk_template_version, qualname_convention_version,
-- last_indexed_commit, head_at_index, indexed_commit_count

CREATE TABLE commits (
  hash TEXT PRIMARY KEY,
  author_name TEXT, author_email TEXT,
  authored_at INTEGER, authored_tz_offset INTEGER,
  committed_at INTEGER,
  topo_order INTEGER NOT NULL,          -- ordering source of truth
  parent_hashes TEXT, is_merge INTEGER NOT NULL DEFAULT 0,
  message TEXT,                          -- raw, for display
  insertions INTEGER, deletions INTEGER, files_changed INTEGER
);
CREATE INDEX ix_commits_topo ON commits(topo_order);

CREATE TABLE files (
  id INTEGER PRIMARY KEY, path TEXT NOT NULL UNIQUE, language TEXT
);

CREATE TABLE function_lineage (
  lineage_id INTEGER PRIMARY KEY, language TEXT
  -- NO current_path/current_name: derived, see view below
);

CREATE TABLE lineage_derivation (
  child_lineage_id INTEGER, parent_lineage_id INTEGER,
  kind TEXT,                             -- split | merge | copy
  at_commit TEXT REFERENCES commits(hash),
  PRIMARY KEY (child_lineage_id, parent_lineage_id, at_commit)
);

CREATE TABLE function_touches (
  touch_id INTEGER PRIMARY KEY,
  lineage_id INTEGER NOT NULL REFERENCES function_lineage(lineage_id),
  commit_hash TEXT NOT NULL REFERENCES commits(hash),
  file_id INTEGER NOT NULL REFERENCES files(id),
  qualified_name TEXT NOT NULL,
  normalized_signature TEXT, param_count INTEGER,
  start_line INTEGER, end_line INTEGER,    -- citation payload (I4)
  change_type TEXT NOT NULL,               -- added|modified|removed|renamed|moved
  signature_before TEXT, signature_after TEXT,
  fingerprint BLOB,
  chunk_text TEXT NOT NULL,                -- templated, leakage-stripped, redacted
  chunk_text_hash BLOB NOT NULL,           -- dedupe
```

```
  parse_status TEXT NOT NULL,             -- ok|degraded|failed
  match_rule TEXT NOT NULL,               -- which lineage rule fired (auditability)
  match_confidence REAL,
  UNIQUE (lineage_id, commit_hash)        -- enforces one touch per commit per lineage
);
CREATE INDEX ix_touch_lineage ON function_touches(lineage_id, commit_hash);
CREATE INDEX ix_touch_commit ON function_touches(commit_hash);
CREATE INDEX ix_touch_name ON function_touches(qualified_name);

CREATE VIEW lineage_current AS
SELECT l.lineage_id, t.file_id, t.qualified_name, t.commit_hash
FROM function_lineage l JOIN function_touches t ON t.lineage_id = l.lineage_id
WHERE t.touch_id = (SELECT t2.touch_id FROM function_touches t2
                    JOIN commits c ON c.hash = t2.commit_hash
                    WHERE t2.lineage_id = l.lineage_id
                    ORDER BY c.topo_order DESC LIMIT 1);

CREATE TABLE commit_file_changes (
  commit_hash TEXT REFERENCES commits(hash),
  file_id INTEGER REFERENCES files(id),
  change_type TEXT, insertions INTEGER, deletions INTEGER,
  PRIMARY KEY (commit_hash, file_id)
);

CREATE TABLE co_change_stats (                -- see Part 4 for semantics
  file_a INTEGER, file_b INTEGER,
  support INTEGER NOT NULL,                   -- co-occurrence count (filtered)
  count_a INTEGER NOT NULL, count_b INTEGER NOT NULL,
  confidence_ab REAL, confidence_ba REAL, lift REAL,
  decayed_support REAL,
  followup_fix_count INTEGER,
  PRIMARY KEY (file_a, file_b), CHECK (file_a < file_b)
);

CREATE TABLE call_edges (                     -- TEMPORAL validity (review fix)
  caller_lineage_id INTEGER NOT NULL,
  callee_lineage_id INTEGER NOT NULL,
  valid_from_commit TEXT NOT NULL REFERENCES commits(hash),
  valid_to_commit TEXT,                       -- NULL = still present at HEAD
  resolution_kind TEXT NOT NULL,              -- intra_file|same_package|unresolved
  confidence REAL,
  PRIMARY KEY (caller_lineage_id, callee_lineage_id, valid_from_commit)
);

CREATE TABLE embeddings (
  touch_id INTEGER PRIMARY KEY REFERENCES function_touches(touch_id),
  row_offset INTEGER NOT NULL
);

CREATE TABLE explain_cache (                  -- isolated; never an index input (I5)
  cache_key TEXT PRIMARY KEY, model TEXT NOT NULL,
  prompt_version TEXT NOT NULL, text TEXT, created_at INTEGER
);
```

SQLite operational notes: - `PRAGMA journal_mode=WAL`, `synchronous=NORMAL` during bulk index, `foreign_keys=ON`, `busy_timeout`. - Single writer thread with a bounded channel; workers parallelize parse/embed and send rows. Concurrent SQLite writers will give you `SQLITE_BUSY` under load. - Batch inserts in transactions of ~5-10k rows; a per-row transaction is 100x

slower. - `schema_version` check on open, with a refusal message telling the user to reindex. You will change this schema. - Index atomically: write to `.gitctx/index.db.tmp`, then rename. A crashed index must never leave a half-populated DB that silently serves wrong results.

Step 3.6 Retrieval: Tantivy + dense + RRF

- **Tantivy** with a code-aware tokenizer: split on camelCase/snake_case boundaries *and* keep the original token, lowercase, no stemming for identifiers (stemming `Session` and `sessions` together is fine; stemming identifiers generally is not). Index fields: `chunk_text`, `qualified_name` (boosted), `path`, `message`.
- **Dense:** `fastembed-rs` MiniLM (or the Phase 0 winner) → single mmap'd `f32` array, L2-normalized at write. Flat brute-force SIMD dot product. At 200k chunks × 384 dims that is ~300 MB and tens of milliseconds — measure it and only then consider `hnsw_rs`. Store `row_offset` in SQLite so the vector file and DB stay consistent; validate row count against `COUNT(*)` on open.
- **RRF fusion:** `score = Σ 1/(k + rank_i)`, k=60. Report k sensitivity briefly (k ∈ {10, 60, 120}) so the choice is not arbitrary.
- **Exact-identifier path:** a query that looks like an identifier (`fetch_user`, `ApiClient.get`) must hit a deterministic lookup on `qualified_name` and be merged into results with high priority. Dense retrieval is genuinely bad at exact symbols; do not let a demo fail on `gitctx search fetch_user`.
- **Dedupe** by `chunk_text_hash` and cap results per commit (e.g. 3 touches) so one large commit cannot fill the top-k.
- **Chain assembly:** for each hit, fetch the full lineage chain by indexed lookup on `(lineage_id)` ordered by `topo_order`. This is the deterministic-completeness claim; it must be a single indexed query, not a graph walk.

Step 3.7 Secret redaction (scoped honestly)

Redact before storage and before embedding: high-entropy strings above a length threshold, plus a small cited ruleset of well-known key prefixes. Documented as *"reduces accidental exposure in tool output; not a security control and not a detection benchmark."* No claimed detection rate. Test with a fixture repo containing planted fake secrets, and assert they are absent from `chunk_text`, from the Tantivy index, and from CLI output. Note that redaction slightly perturbs embeddings — mention it, do not hide it.

Step 3.8 CLI (Phase 1 subset)

`gitctx index [--incremental]`, `search`, `evolution`, `show`, `doctor`. **JSON first**, with a versioned envelope:

```
{"gitctx_version":"0.3.1","schema_version":4,"config_hash":"...",
 "command":"search","query":"...","warnings":[],"results":[...]}
```

`gitctx doctor` is a genuinely valuable 2-hour investment: reports schema/config hash match, index staleness vs. HEAD, parse-status distribution, vector/DB row-count consistency, and exclusion fractions. It turns "it gave weird results" into a diagnosis.

Phase 1 definition of done: the Phase 0 harness runs unmodified against the Rust binary and reproduces or beats the Python numbers, within CI. If it does not, the discrepancy is a

bug in the Rust implementation and must be found before proceeding — this is the single most valuable checkpoint in the plan.

Part 4. Phase 2 — Graph and history statistics (5-6 days)

Step 4.1 Co-change statistics done correctly

Raw co-occurrence counts are dominated by noise. Required treatment:

- **Size filtering:** exclude commits touching more than `cochange_max_files` (default 25, tuned and reported) from co-change accounting. A single `black` reformat over 900 files contributes ~400k pairs and swamps every genuine coupling. Report how many commits were excluded and what fraction of pairs they would have contributed — that delta is a nice finding in itself.
- **Normalized measures, not counts:** support, confidence `P(B|A)` and `P(A|B)` (asymmetric — keep both), and lift. Rank by confidence with a minimum-support floor (e.g. support ≥ 5) to suppress the coincidental-pair long tail.
- **Time decay:** exponential half-life (default 18 months, configurable). Coupling from 2014 is weaker evidence than 2025. Report metrics with and without decay.
- **Exclusions:** bots, vendored paths, lockfiles, generated files, merges.
- **Define "fix" operationally** for `followup_fix_count`, in priority order: (a) commit linked to an issue labeled bug, (b) commit is a revert of the earlier commit, (c) message matches a stated regex. Prefer (a)/(b); label (c) explicitly as weak evidence when used.

Step 4.2 The graph

`petgraph` typed multigraph built from SQLite at load: commit→file, commit→lineage, lineage→lineage (call edges, time-filtered), file↔file (co-change). Keep SQLite as the source of truth and the graph as a derived in-memory structure — never the reverse.

Pitfall: loading a 50k-commit repo's full graph may cost seconds and hundreds of MB. Load lazily per command, or build only the subgraph a command needs. Measure and record.

Step 4.3 `impact` and evaluation against a real baseline

`gitctx impact <path|function>` returns: co-changed files ranked by confidence with support, historical fix-follow rate, top contributors, and churn/recency.

Validation — temporal holdout: train co-change on commits with `authored_at < T`, evaluate against commits $\geq$ T. Sweep T (e.g. 70%, 80%, 90% of history) so results are not an artifact of one cut point.

Baselines — this is where the review's point bites hardest: 1. Random / frequency prior (predict the most-frequently-changed files). The floor. 2. Same-directory heuristic. 3. **`code-maat`** (already in your sources) — an established tool doing exactly this. Comparing against it is far stronger than comparing against a strawman. If you match it, your framing is "reproduces established logical-coupling findings within an agent-facing interface"; if you beat it (likely via decay + normalization + function granularity), that is a real result. Either outcome is defensible; having no comparison is not.

Metrics: Precision@5/;, Recall@10, MRR, plus a lift-over-frequency-prior number, per repo and per holdout point.

Pitfall to pre-empt: co-change prediction is partly self-fulfilling and partly trivial (a file's own tests always co-change). Report with and without test files, and say so.

Part 5. Phase 3 — Agent surface and human output (5-6 days)

Step 5.1 MCP server

`rmcp` over stdio. Tools: `search_history`, `function_evolution`, `change_impact`, `who_knows_this`, `commit_detail`.

Requirements and pitfalls: - Tool return shapes must be **identical** to the corresponding `--json` payloads. Enforce with a shared serde type and a test asserting equality; two drifting shapes is a guaranteed late bug. - **Token budgeting.** Returning 20 chunks with full diffs will blow the calling agent's context. Every tool needs a `max_results` and a `detail` level (`compact` = facts + citations; `full` = includes diff text), with `compact` as the default and a documented approximate token cost per result. This is the most common real-world failure of MCP tools and is worth a paragraph in your README. - **stdout discipline.** MCP speaks JSON-RPC on stdout. Any stray `println!` or progress bar corrupts the stream. Route all logging to stderr, and add a test that asserts stdout contains only valid framed JSON. - **Stale index handling.** If HEAD moved since indexing, return a `warnings` entry rather than silently serving stale facts, and include `head_at_index` in every response. - **Concurrency:** multiple readers on WAL SQLite are fine; ensure the vector mmap is opened read-only and shared. - **Timeouts and cancellation:** long queries must be interruptible; an agent that hangs is worse than one that errors.

Step 5.2 Skills and init

`gitctx init` writes skill profiles (`.agents/skills`, `.claude/skills`), an `AGENTS.md` stanza, and config. Must be **idempotent** and non-destructive: detect existing stanzas by a marker comment, never clobber user edits, support `--dry-run` and `--force`. Test by running it twice and asserting no diff on the second run.

The SKILL.md must state *when not to use* each tool (e.g. do not use `search_history` for current code structure — that is codanna's job). Agents follow negative guidance surprisingly well, and it reinforces your positioning.

Step 5.3 Human-facing output — budget real time

The structured-output pivot created this gap; a wall of JSON is a bad demo and a bad defense.

- `--format table` (default for TTY), `--format json` (default when piped), `--format markdown`.
- `evolution` should render as a chronological timeline: date, short SHA, author, one-line message, signature delta, `+/-` counts.
- Respect `NO_COLOR`, terminal width, and non-TTY detection. Never emit ANSI into a pipe.
- Pager behavior, and `--no-pager`.

Verification: golden-file snapshot tests for each format against fixture repos, so output changes are deliberate.

Part 6. Cross-cutting verification — the tests that make the claims credible

6.1 Pinned, versioned evaluation corpus

`eval/corpus.lock.toml` with repo URLs and exact SHAs; harness refuses to run on mismatch. Commit the harvested query/label dataset (or a script plus a hash of it if licensing forbids redistribution). Without pinned SHAs, your numbers are unreproducible the moment upstream moves — 20 minutes of work separating "results" from "anecdotes."

6.2 Determinism test (proves I2)

Index the same repo twice into separate DBs. Normalize (strip autoincrement IDs and timestamps; canonically order rows) and assert byte-identical dumps for `function_touches`, lineage assignments, and `co_change_stats`. Additionally assert identical top-10 for a fixed 50-query set. Sources of nondeterminism to hunt when it fails: `HashMap` iteration order, `rayon` work-stealing affecting insert order or greedy matching, float summation order, `tokio` task ordering, and tie-breaking in ranking (always break ties by a stable key such as `topo_order` then `touch_id`).

6.3 Incremental equivalence test (proves I3)

Index commits 1..N fully; separately index 1..(N−10) then incrementally add the last 10. Assert equivalence. This catches the bug class that silently ruins incremental indexers — typically lineage state that depended on seeing the whole history, or co-change denominators not being updated.

Related pitfalls to test: force-push/rebase invalidating `last_indexed_commit` (detect by checking the recorded commit is still an ancestor of HEAD; if not, require a full reindex), and amended commits.

6.4 Pre-registered failure taxonomy

Before looking at results, define the buckets you will classify misses into: query-vocabulary mismatch, chunk granularity, lineage break, commit genuinely uninformative, label incomplete, truncation, exclusion filter. Then report counts per bucket. This converts your failure-analysis section from apology into method, and it directly pre-empts the harshest defense questions.

6.5 Property and fuzz tests

- Property: for any lineage, the assembled chain is strictly increasing in `topo_order`, contains no duplicate commits, and every element cites a real commit.
- Property: `co_change_stats` symmetric pairs satisfy `file_a < file_b` and `support ≤ min(count_a, count_b)`.

- Fuzz: run the indexer over randomly generated small repos (random renames, moves, deletions, merges) asserting no panics and no constraint violations. This finds the many-to-many matching bugs that hand-written tests miss.

6.6 Performance budgets as pass/fail CI tests

Metric	Budget	Measured how
Cold index, 10k commits	< 10 min, single machine	`hyperfine`, medium corpus, recorded hardware
Cold index, 50k commits	< 60 min	large corpus, reported not gated
Incremental, 1 commit	< 2 s	fixture + medium corpus
`search` p95	< 300 ms	200-query replay, warm
`impact` p95	< 500 ms	includes graph load
MCP round trip	< 500 ms	stdio harness
DB size	< 100 MB / 10k commits	recorded per corpus
Peak RSS during index	< 2 GB	`/usr/bin/time -v`

Record hardware. Report flat-cosine vs. HNSW measurements even if you never adopt HNSW — "we measured that we did not need the fancy index" is a strong slide.

Part 7. Phase 4 — Evaluation report and defense artifacts (5 days)

Deliverable: `eval/REPORT.md`, generated by `cargo xtask eval` so it is reproducible, containing:

1. Corpus table with pinned SHAs, exclusion fractions, bot filtering, parse-status distribution.
2. **Harvester precision** from the 30-pair audit — your honest metric ceiling.
3. Retrieval: full grid (document variants × retrievers), Set A and Set B **reported separately**, per repo and pooled, with bootstrap CIs, adjudicated-miss rates, RRF k sensitivity, truncation rates.
4. Lineage: per-transformation-class precision/recall from synthetic mutation; chain-level accuracy stratified by length; `git log -L` agreement with its blindness quantified (including the fraction of lineages it cannot express).
5. `impact`: temporal holdout at 3 cut points vs. frequency prior, same-directory heuristic, and `code-maat`; with/without test files; with/without decay.
6. Performance against every budget, with hardware.
7. Ablations that answer "which part earns its keep": template on/off, hybrid vs. dense-only vs. BM25-only, decay on/off, size-filter on/off.
8. **Failure analysis** with pre-registered buckets and counts.
9. **Limitations and non-goals**, explicit: Python-only, first-parent traversal, intra-file/same-package call resolution only, co-change is correlational not causal, redaction is not a security control, labels are incomplete lower bounds.

README additions: the positioning line, architecture diagram, an asciinema demo, quickstart, an explicit non-goals section, and a one-paragraph "how to reproduce every number in the report."

Part 8. Phase 5 — Current-state layer (deferred, and the designated slip absorber)

Only if Phases 0-4 are complete and the schedule has runway. Ordered by value per unit risk:

1. **Second language (Rust or TS), extraction only** — 3-4 days. Breadth you can partially evaluate.
2. **Intra-file + same-package call graph** — 4-5 days. Everything else marked `unresolved`. Report `intra_file` / `same_package` / `unresolved` proportions and hand-verify 30-50 edges for precision by resolution kind. Cross-file resolution in Python (dynamic dispatch, `import x as y`, re-exports, monkeypatching, duck typing) is genuinely hard; do not promise it. Gate: if `cross_file` coverage < 50%, ship intra-file only and report coverage honestly.
3. **`symbols find` / `symbols callers`** — 2 days, on top of the above.
4. **mmap symbol cache** — 1-2 days, and only *after* benchmarking the SQL baseline at target repo size. Report both numbers. If SQL meets budget, do not build it, and say why.
5. **Doc-RAG / named collections — cut.** It serves none of the three thesis claims, is a separate product, and is the highest-cost/lowest-defense-value item in the plan. If you want it, it is Phase 6.

Part 9. Phase 6 — Optional extras

`--explain` + `explain_cache` (gated behind a cargo feature, model and prompt version recorded, never an index input); `--watch` (honestly a background-process/cache-invalidation/concurrent-read problem, not a flag); `npx` wrapper (per-target-triple cross-compilation CI for linux-x64/arm64, macos-arm64/x64, windows-x64 — a day or two, plus npm `optionalDependencies` platform packages and a postinstall shim); third/fourth language; validated ML risk model beyond the co-change heuristic.

Part 10. Schedule, slack, and pre-committed cuts

Phase	Work	Duration
−1	Scaffolding, fixtures, corpus selection, harvesting	3 days
0	Retrieval thesis validation (Python)	4 days
1	Rust core engine	12 days
2	Graph and statistics	6 days
3	Agent surface and human output	6 days
4	Evaluation report and docs	5 days
—	**Buffer (25%)**	**9 days**
	Total to defensible product	**~45 working days (~9 weeks)**
5	Current-state layer	+10-14 days if runway

Pre-committed cut order (decide now, not under pressure): doc-RAG → third/fourth language → mmap symbol cache → cross-file call resolution → `--watch` → `npx` → second language. Never cut: Phase 0, lineage validation, ground-truth non-circularity, determinism tests, the failure-analysis section.

Gate table

Checkpoint	Measure	Action if failed
End Phase −1	Traceable issues ≥ 150 per repo; harvester precision ≥ 0.85	Change repos or fix harvester before proceeding
End Phase 0	ΔRecall@10 (template vs. BM25 and vs. raw diff), CI excludes 0	Template loses → demote dense to secondary; thesis becomes lineage + impact
Phase 1 midpoint	Synthetic M2/M3 (cross-file move) recall	< 0.6 → restrict claim to intra-file lineage, state it, stop investing
End Phase 1	Rust reproduces Phase 0 numbers	Discrepancy is a bug; do not proceed until found
End Phase 1	Cold index 10k commits vs. budget	> 3x → cap corpus size and document the limit rather than optimizing
End Phase 2	`impact` vs. code-maat	Not better → reframe as reproducing established findings in a new interface
End Phase 3	MCP compact-mode token cost per call	Too high → reduce default `max_results`, add summarized mode
End Phase 5	`cross_file` resolution coverage	< 50% → ship intra-file only, report coverage

Part 11. Risk register — the things most likely to actually hurt

Risk	Likelihood	Impact	Mitigation	Detected by
Dense retrieval on diffs underperforms BM25	High	High	Phase 0 gate; templated docs; hybrid RRF; code-aware model as second lever	Phase 0 grid
Label leakage inflates BM25 baseline	High	High	Strip trailers/ refs before indexing; log strip counts	Leakage audit; suspiciously high BM25
Ground truth measures the wrong query distribution	High	High	Set B why-shaped queries reported separately	A/B divergence in report
Cross-file move lineage is unreliable	Medium-High	High	Scoped cross-file fingerprint search; per-class synthetic metrics; fall back to stated intra-file claim	Synthetic M2/ M3
Chain accuracy collapses over long chains	Medium	High	Chain-level metric stratified by length; raise τ precision	Chain-level eval
Merge/non-linear history corrupts stats	Medium	High	First-parent default; evil-merge detection; `is_merge`	Merge fixtures; commit-count cross-check
Reformat commits swamp co-change	High	Medium	Size filter + normalization + decay	Exclusion-fraction report
Index time unacceptable on large repos	Medium	Medium	Blob-OID parse cache; rayon; batched writes; cap diff size	Perf CI
Nondeterminism from parallelism	Medium	High	Stable tie-breaks; single writer; determinism test	Double-index test
Incremental index diverges from full	Medium	High	Equivalence test; ancestry check on `last_indexed_commit`	6.3
Schema v2 forces full rebuild for all users	Medium	Medium	Version-aware open; auto-rebuild; clear error	Version-mismatch test

Part 12. The three claims, and exactly how each is proven

1. **Deterministic template-rendering makes natural-language retrieval over git history work.** Proven by the Phase 0 ablation grid with paired bootstrap CIs, on non-circular linked-issue ground truth with leakage stripped, reported separately for issue-shaped and why-shaped queries, with adjudicated misses and a pre-registered decision rule. Re-verified in Rust at the end of Phase 1.
2. **Function identity can be tracked across renames and moves well enough to assemble complete chronological chains.** Proven by synthetic mutation testing with per-transformation precision/recall including a false-positive class (M8), reported at chain level and stratified by chain length, with `git log -L` as a secondary oracle whose blindness is quantified rather than hidden.
3. **The evaluation is honest.** Proven by pinned corpus SHAs, published harvester precision, stated baselines including an established tool (`code-maat`), determinism and incremental-equivalence tests, performance budgets as pass/fail, a pre-registered failure taxonomy, and an explicit limitations/non-goals section.

Everything else in this plan — Rust, MCP, skills, packaging, multi-language, symbol caches, `--explain` — is real engineering work that makes the project usable and impressive. But it is no longer carrying the argument. These three are.

You · Aug 16, 14:27

Which parts should get the deepest treatment? → Everything, evenly

How should the plan be structured? → Hybrid

What extras should be included inline? → Rust/SQL sketches, Test & CI matrix, Risk register, Defense/README prep

How long should the plan be? → Extra long

GPAI · Aug 16, 14:27

I'll create a Deep Explain artifact containing the complete implementation plan — a hybrid phase-plus-workstream structure with even depth across all parts, including inline Rust/SQL sketches, a full test and CI matrix, an expanded risk register, and defense/README preparation material.